Zadanie 1 - opis rozwiązania

PWR Racing Team - Rekrutacja Electronics

Projekt wykonano dla mikrokontrolera STM32G473CET3 z zewnętrznym rezonatorem 16 MHz. Poniżej zestawiono najważniejsze obliczenia oraz sposób doboru parametrów użytych w konfiguracji.

1. Zegar systemowy

Dostępny jest rezonator 16 MHz, a zegar systemowy ma mieć 170 MHz. Dla wyjścia PLL używanego jako SYSCLK zależność ma postać:

```
f_SYSCLK = (16 MHz / M) * N / R
```

Parametry M, N i R trzeba więc dobrać razem tak, aby po podstawieniu otrzymać dokładnie 170 MHz. R jest końcowym dzielnikiem częstotliwości PLL. Zacząłem od najmniejszej dostępnej wartości R = 2; jeżeli dla niej da się znaleźć poprawne całkowite M i N, nie ma potrzeby zwiększać tego dzielnika.

Po podstawieniu R = 2 równanie można zapisać jako:

```
170 = (16 / M) * N / 2
N = 21.25 * M
```

N musi być liczbą całkowitą, dlatego przeliczyłem kolejne wartości M. Dla M = 1 otrzymujemy N = 21.25, dla M = 2 jest 42.5, dla M = 3 jest 63.75, a dla M = 4 wychodzi dokładnie N = 85. Ta para spełnia wymaganie i daje poprawną częstotliwość wejściową PLL równą 4 MHz.

```
16 MHz / 4 = 4 MHz
4 MHz * 85 / 2 = 170 MHz
```

Po ustawieniu SYSCLK na 170 MHz prescalery AHB, APB1 i APB2 pozostawiłem równe 1, ponieważ nie było potrzeby dodatkowo obniżać częstotliwości magistral.

2. TIM6 i miganie LED z częstotliwością 5 Hz

Przy 5 Hz jeden pełny okres migania trwa 200 ms. LED jest przełączany z 0 na 1 albo z 1 na 0 przy każdym przerwaniu, więc przerwanie musi występować dwa razy w jednym okresie. Potrzebny odstęp między przerwaniami wynosi zatem 100 ms.

```
T_LED = 1 / 5 Hz = 200 ms
T_przerwania = 200 ms / 2 = 100 ms
```

Pozostawał dobór prescalera i ARR. Zamiast dobierać wartości metodą prób, najpierw wybrałem wygodną częstotliwość licznika: 10 kHz. Oznacza to jeden krok licznika co 100 us. Przy zegarze TIM6 równym 170 MHz trzeba więc podzielić go przez 17000. Ponieważ rejestr prescalera przechowuje wartość o jeden mniejszą od rzeczywistego dzielnika, otrzymujemy PSC = 16999.

```
PSC = 170 MHz / 10 kHz - 1 = 16999
```

Do uzyskania 100 ms potrzeba wtedy dokładnie 1000 kroków po 100 us. Timer liczy od zera, więc ARR = 999. Taki dobór daje proste, łatwe do sprawdzenia wartości i nie wymaga żadnych przybliżeń.

```
100 ms / 100 us = 1000 kroków -> ARR = 999
```

3. Watchdog

W zadaniu nie narzucono czasu watchdogu, więc przyjąłem około 2 s. Jest to znacznie dłużej niż normalny czas wykonania jednej iteracji głównej pętli, dlatego watchdog nie powinien resetować układu podczas prawidłowej pracy. Jednocześnie po zawieszeniu programu reakcja nie jest przesadnie opóźniona.

Wybrałem IWDG, ponieważ pracuje z niezależnego zegara LSI. Dla nominalnych około 32 kHz prescaler 32 daje około 1 kHz, czyli jeden krok licznika trwa w przybliżeniu 1 ms. Wtedy wartość Reload = 1999 daje około 2000 kroków, czyli około 2 s.

```
32 kHz / 32 ~= 1 kHz
(1999 + 1) / 1 kHz ~= 2 s
```

Window ustawiłem na maksymalną wartość 4095. W tym projekcie zależy mi tylko na wykryciu braku odświeżenia, a nie na wymuszaniu konkretnego momentu, w którym odświeżenie może nastąpić. Trzeba też pamiętać, że LSI nie jest precyzyjnym źródłem, więc 2 s jest wartością przybliżoną.

4. FDCAN - dobór 1 Mbit/s

Zegar kernela FDCAN wynosi 170 MHz, a wymagana prędkość to 1 Mbit/s. Oznacza to, że cały dzielnik od zegara FDCAN do pojedynczego bitu musi mieć wartość 170. Dzielnik ten jest iloczynem nominalnego prescalera i liczby time quanta przypadających na jeden bit.

```
170 MHz / 1 MHz = 170
```

Wybrałem Nominal Prescaler = 10, co daje 17 MHz dla time quantum. Wtedy na jeden bit potrzeba dokładnie 17 tq. Taka liczba jest wygodna, bo pozwala ustawić punkt próbkowania w typowym obszarze około 80% długości bitu, a jednocześnie wszystkie wartości pozostają całkowite.

```
170 MHz / 10 = 17 MHz
17 MHz / 17 tq = 1 Mbit/s
```

Jeden tq jest zajęty przez segment synchronizacji. Pozostałe 16 tq podzieliłem na TSEG1 = 13 i TSEG2 = 3. Punkt próbkowania wypada wtedy po 14 z 17 tq, czyli około 82.35%. Nie jest to jedyna poprawna konfiguracja; ten podział został wybrany dlatego, że daje dokładny bitrate i rozsądne położenie punktu próbkowania.

```
sample point = (1 + 13) / 17 = 82.35%
```

SJW = 3 jest zgodne z TSEG2 = 3 i pozwala kontrolerowi skorygować synchronizację maksymalnie o 3 tq. Tryb ramki pozostaje Classical CAN, ponieważ w zadaniu nie ma potrzeby używania fazy danych CAN FD.

5. Ramka z uptime

Cała informacja ma zmieścić się w dwóch bajtach, dlatego naturalnym typem dla wartości uptime jest uint16_t. Jako jednostkę przyjąłem sekundy. Dzięki temu wymaganie dokładności 1 s jest spełnione bez dodatkowego skalowania, a wartość można uzyskać bezpośrednio z HAL_GetTick().

```
uptime_seconds = (uint16_t)(HAL_GetTick() / 1000U)
```

Dwa bajty wysyłam w kolejności MSB first. Nie wynika to z konieczności sprzętowej - jest to jawna decyzja dotycząca formatu danych. Dzięki temu odbiornik nie musi zgadywać, w jakiej kolejności należy złożyć dwa bajty w jedną liczbę.

```
data[0] = starszy bajt, data[1] = młodszy bajt
```

Przykładowo dla wartości 0x1234 payload ma postać [0x12, 0x34]. Wadą dwóch bajtów jest zawinięcie po 65535 s, czyli po około 18.2 h. Przy narzuconej długości ramki jest to jednak bezpośrednia konsekwencja wybranego formatu.

6. Wysyłanie co 1 s

Do odmierzania jednej sekundy użyłem HAL_GetTick(), ale bez HAL_Delay(). Opóźnienie blokujące zatrzymałoby główną pętlę na cały czas oczekiwania. Zamiast tego pętla cały czas pracuje, a transmisja następuje dopiero po stwierdzeniu, że od poprzedniej transmisji minęło co najmniej 1000 ms.

```
if ((now - last_uptime_tx) >= 1000U)
```

Odejmowanie jest celowe również ze względu na przepełnienie HAL_GetTick(). Dla typu unsigned różnica current - previous zachowuje poprawny czas, który upłynął przez moment zawinięcia licznika. Dzięki temu nie trzeba osobno obsługiwać przejścia z maksymalnej wartości uint32_t do zera.